TERRAFORM

INTERVIEW MASTER GUIDE

200 Questions | Beginner to Advanced | Real Project Answers | Azure + DevOps

Core Concepts	**State Management**	**Modules**	**CI/CD**
DevSecOps	**Azure-Specific**	**Troubleshooting**	**Real Project**

Prepared for: Dilip Bhai | All the Best! ■

SECTION 1 Core Concepts — IaC & Terraform Basics

Q1. What is Terraform?

Terraform ek declarative Infrastructure as Code (IaC) tool hai jo cloud resources ka pura lifecycle manage karta hai — state file maintain karke. HCL configuration padh kar Azure jaise cloud providers se resources create, update, aur delete karta hai.

> ***Interview Tip:*** *In my project, I use Terraform to provision Azure resources — VMs, storage, networking — integrated with Azure DevOps for automated deployments.*

Q2. What is Infrastructure as Code (IaC)?

IaC ka matlab hai infrastructure ko code se manage karna — manual steps ki jagah. Automation, consistency, version control, aur repeatability milti hai across environments.

Q3. What is a Provider?

Provider ek plugin hai (jaise azurerm) jo Terraform ko cloud platforms se interact karne deta hai. HCL code ko cloud provider ke API calls mein translate karta hai.

Q4. Resource vs Data Source — Key Difference?

- resource = naya cloud component banata hai (VM, storage, VNet)
- data "azurerm_..." = existing resource ka reference leta hai — managed nahi karta
- Example: data source se existing Key Vault ka URI read karo

Q5. Why use Terraform?

Automation, consistency, scalability, aur manual errors reduce karta hai. Cloud-agnostic hai, 100+ providers support karta hai, aur remote state se team collaboration enable hoti hai.

SECTION 2 Terraform Commands — Har Command ka WHY

Command	Kya Karta Hai	Pro Tip
`terraform init`	Plugins download, backend setup karo	Pehla command — hamesha run karo
`terraform plan`	Dry-run — kya change hoga dikhao	Production mein -out flag use karo
`terraform apply`	Changes apply karo	-auto-approve KABHI production mein nahi
`terraform destroy`	Sab kuch delete karo	prevent_destroy se critical resources bachao
`terraform fmt`	Code format karo	Commit se pehle run karo
`terraform validate`	Syntax check — cloud connect nahi karta	Fast local check
`terraform plan -out=tfplan`	Plan file save karo	CI/CD approval flow ke liye essential
`terraform state mv`	Resource rename karo state mein	Destroy-recreate avoid karta hai

`terraform state rm`	Terraform bhool jaye resource — delete nahi	Resource Azure mein zinda rehta hai
`terraform import`	Existing resource ko Terraform mein laao	Phir HCL likho, plan se verify karo
`terraform init -upgrade`	Provider plugins upgrade karo	Dev mein pehle test karo
`terraform force-unlock`	Stuck state lock remove karo	Tabhi use karo jab sure ho koi run nahi

Q: terraform plan -out vs terraform apply -auto-approve?

- plan -out=tfplan → reviewed plan file save hoti hai — exactly yahi apply hogi
- apply -auto-approve → bina review ke changes apply — NEVER production mein
- Best practice: plan → review/approve → apply saved plan

> ***Interview Tip:*** *Nothing reaches production without security scan, plan review, and manual approval gate.*

SECTION 3 State Management — Sabse Important Topic

Q: What is Terraform State?

Ek JSON file (terraform.tfstate) jo configuration ko real-world resources se map karti hai. Terraform isko use karta hai track karne ke liye ki kya create ho chuka hai aur updates/deletions plan karne ke liye.

Note: State file manually KABHI edit mat karo — hamesha terraform state commands use karo.

Q: Remote Backend kyon use karte ho? (Must-know answer)

Team mein local state dangerous hai — share nahi hoti, locking nahi hoti.

- Azure Blob Storage = remote backend (mere project mein use hota hai)
- Blob lease = state locking — simultaneous runs prevent karta hai
- Versioning + soft-delete = automatic backup if state corrupt ho
- RBAC = sirf pipeline service principal ko read/write access

> ***Interview Tip:*** *In my project, we use Azure Blob Storage as remote backend with blob lease locking — prevents conflicts when multiple engineers work simultaneously.*

Q: What is Drift? How do you handle it?

Drift tab hota hai jab koi manually Azure mein kuch change kar de — real state aur Terraform configuration mismatch ho jaati hai.

- Detect: terraform plan chalao — drift dikhayega
- Fix: terraform apply karo — desired state restore hogi
- Prevent: CI/CD pipeline se sab changes jayein — koi manual change nahi
- Monitor: Daily scheduled plan job jis se drift alert mile

Q: State file corrupt/lost ho jaye to?

- Azure Blob versioning se backup restore karo
- Ya terraform import se har resource wapas laao
- isliye hamesha versioning + soft-delete enable karo storage account pe

Q: What is dependency lock file (.terraform.lock.hcl)?

Ensures consistent provider versions across team members aur CI/CD runs. HAMESHA Git mein commit karo — ye file ensure karti hai sab log same provider version use karein.

SECTION 4 Variables, Outputs & Locals

Variable Precedence (Lowest → Highest):

Priority	Source	Explanation
1 (Lowest)	`TF_VAR_name env vars`	Environment variables
2	`terraform.tfvars`	Auto-loaded file
3	`*.auto.tfvars`	Auto-loaded by name pattern
4 (Highest)	`-var / -var-file flags`	CLI flags — HAMESHA jeetate hain

Q: Variable vs Local vs Output — Key Difference?

- Variable = external input — user ya pipeline provide karta hai (flexible, reusable)
- Local = internal computed value — module ke andar rehta hai, repetition avoid karta hai
- Output = result bahar expose karta hai — doosre modules ya pipelines use kar sakte hain
- Sensitive variable = secrets ko logs/plan output mein print hone se rokta hai

Q: count vs for_each — Kab kya use karein?

- count = N identical resources banata hai (number se)
- for_each = unique naam/config wale resources — map/set se
- for_each PREFER karo — specific item add/remove easy hota hai
- Agar count use karo aur beech mein ek delete karo — sab shift ho jaate hain!

> ***Interview Tip:*** *In my project, I always use for_each for NSGs, storage accounts, and VMs — easier to manage per-resource changes.*

SECTION 5 Modules & Project Structure

Q: Modules kaise use karte ho? (Complete Project Answer)

Modules reusable Terraform code ke blocks hain — ek baar likho, sab jagah use karo.

- Maine banaye hain: VNet, Key Vault, App Service, VM, Storage modules
- Har module: apna variables.tf, outputs.tf, main.tf
- Dev, test, prod — teeno same module call karte hain, sirf values alag
- Module versions pinned hain — existing envs break nahi hote upgrade se

> ***Interview Tip:*** *In my project, I have created reusable modules for Azure VNet, Key Vault, App Service, VM, and storage.*

Project Folder Structure:

```
infra/
■■■ modules/
■ ■■■ vnet/ (main.tf, variables.tf, outputs.tf)
■ ■■■ keyvault/
```

```
■ ■■■ vm/
■ ■■■ storage/
■■■ environments/
■■■ dev/ (backend.tf, main.tf, dev.tfvars)
■■■ test/
■■■ prod/ (backend.tf, main.tf, prod.tfvars)
```

Q: Monolithic vs Modular Terraform?

- Monolithic = single large file — team projects ke liye worst practice
- Modular = organized reusable modules — clear inputs/outputs, team-friendly
- Modular: testing easy, blast radius chhota, ownership clear

---

SECTION 6 Lifecycle & Dependencies

Lifecycle Option	Kya karta hai	Real Use Case
`prevent_destroy`	Accidental delete rokta hai	Key Vault, Storage Account — production
`create_before_destroy`	Pehle naya banao, phir purana delete karo	SSL certificates, VM images — zero downtime
`ignore_changes`	Specific attribute changes ignore karo	Azure Policy jo automatically tags set kare
`replace_triggered_by`	Referenced resource change pe force replace	Resources jo sab changes detect nahi karte

Q: taint kya hai?

DEPRECATED hai — ab terraform apply -replace='resource_address' use karo. Interview mein zaroor batao ki ye outdated hai — shows you are current!

Note: Always mention taint is deprecated — it shows you follow current Terraform practices.

Q: depends_on kab use karo?

Jab Terraform dependency automatically detect na kar sake — for example, ek script jo role assignment complete hone ke baad chalani ho.

> ***Interview Tip:*** *In my project, I use depends_on when a VM extension depends on a Key Vault role assignment completing first.*

SECTION 7 DevSecOps — tfsec, tflint, Checkov

Q: Teen tools ka difference — tflint vs tfsec vs Checkov?

- tflint = code quality checker — deprecated syntax, invalid variable types, provider-specific rules
- tfsec = Terraform HCL security scanner — open ports, unencrypted storage, missing HTTPS
- Checkov = multi-framework policy scanner — CIS, PCI-DSS, SOC2, HIPAA bhi cover karta hai
- Best practice: teeno chalao — broader coverage milti hai

> ***Interview Tip:*** *Shift-left security — catching misconfigurations in the pipeline BEFORE they ever reach cloud infrastructure.*

Common tfsec Findings (Jo interview mein pooche jate hain):

- Storage accounts with public blob access enabled
- Key Vault without soft-delete or purge protection
- NSG rules with unrestricted inbound access (0.0.0.0/0)
- VMs without disk encryption enabled
- Storage accounts without HTTPS-only enforcement
- Missing Azure Monitor diagnostic settings

SECTION 8 CI/CD Pipeline — Complete Azure DevOps Flow

Stage	Step	Why Important
1	`Git Checkout`	Latest code lao
2	`terraform init`	Backend configure, plugins download
3	`tflint`	Code quality aur standards check
4	`tfsec + Checkov`	Security misconfigs — block if critical
5	`terraform plan -out=tfplan`	Saved plan — exactly yahi apply hogi
6	`Manual Approval Gate`	Production ke liye — required review
7	`terraform apply tfplan`	Same reviewed plan apply — no surprises

Q: Saved plan kyon important hai?

Jo plan review + approve hua, wahi apply hota hai — beech mein koi surprise nahi. Team environment mein yeh critical hai. Reviewer exactly jaanta hai kya change hoga.

> ***Interview Tip:*** *Nothing reaches production without code review, security scan, plan review, and manual approval.*

SECTION 9 Azure-Specific Questions

Q: Terraform Azure authentication ke 4 options?

- Azure CLI login — local development ke liye
- Service Principal with client secret — CI/CD (most common)
- Service Principal with certificate — more secure
- Managed Identity — PREFERRED in CI/CD — no secrets to manage!

> ***Interview Tip:*** *In my pipeline, I use a Service Principal configured as Azure DevOps service connection — no secrets stored in code.*

Q: Azure Blob Storage backend configure kaise karo?

- Storage account + container create karo Azure mein
- backend "azurerm" block define karo backend.tf mein
- Specify: resource_group_name, storage_account_name, container_name, key
- terraform init chalao backend initialize karne ke liye
- IMPORTANT: versioning + soft-delete enable karo storage account pe

Q: Azure Key Vault secrets Terraform mein kaise handle karte ho?

data source se existing Key Vault secrets read karo runtime pe. Kabhi secrets tfvars ya code mein store mat karo. Sensitive = true har secret variable pe.

> ***Interview Tip:*** *In my project, all secrets come from Azure Key Vault via pipeline variable group linking — never hardcoded.*

Q: Service Principal ko kaun sa RBAC role chahiye?

- Minimum: Contributor role on subscription ya resource group
- Role assignments ke liye: Owner ya User Access Administrator
- Always apply principle of least privilege — minimum permissions
- Subscription-wide Owner kabhi mat do — resource group scoped prefer karo

---

SECTION 10 — Comparison Questions — Interviewer Favorites

Tool	Category	Key Difference	When to use
Terraform	IaC Provisioning	Declarative, state-managed, cloud-agnostic	Infrastructure create karo
Ansible	Config Management	Procedural, agentless, no state	Software configure karo existing servers pe
ARM Templates	Azure-only IaC	JSON, verbose, no state	Avoid — Terraform better hai
CloudFormation	AWS-only IaC	YAML/JSON, AWS-integrated	Single AWS — else Terraform
Pulumi	IaC (code-first)	Python/TS/Go use karo	Dev-heavy teams jo code prefer karein

Q: Declarative vs Imperative IaC?

- Declarative (Terraform) = WHAT chahiye batao, HOW Terraform figure out karta hai
- Imperative (scripts) = HOW karna hai exactly batao — step by step
- Declarative zyada predictable aur idempotent hota hai

SECTION 11 Troubleshooting & Scenario Questions

Q: Two engineers terraform apply ek saath chalayein?

State locking ek ko rokti hai — doosra wait karta hai. Isliye remote backend with locking ESSENTIAL hai. Bina locking ke state corrupt ho sakti hai.

Q: Pipeline beech mein fail ho jaye?

Root cause fix karo aur pipeline dobara chalao. Terraform state check karta hai — sirf jo missing hai wahi create karta hai. Partial state gracefully handle hoti hai.

Q: Koi resource manually delete kar de Azure mein?

Terraform drift detect karta hai next terraform plan pe aur next apply mein recreate karta hai — lifecycle rules ke hisaab se.

Q: State lock stuck ho jaye?

terraform force-unlock use karo — TABHI jab sure ho koi aur Terraform operation nahi chal raha.

Q: Backend config change ho?

terraform init dobara chalao — Terraform state naye backend pe migrate kar deta hai.

Q: Provider plugin fail ho?

terraform init re-run karo, network check karo, required_providers constraints verify karo, .terraform.lock.hcl conflicts dekho.

SECTION 12 Real Project Answers — Interview Mein Exactly Kaise Bolein

Q: Tell me about yourself / your Terraform project?

I use Terraform to provision Azure resources — VMs, storage accounts, VNets, NSGs, Key Vault, and App Services. I have created reusable modules for each resource type. Terraform is integrated with Azure DevOps pipelines with manual approval gates for production. Remote state is stored in Azure Blob Storage with blob lease locking.

> ***Interview Tip:*** *10 years of combined telecom + DevOps experience — genuinely rare. Own it with confidence!*

Q: Biggest challenge in Terraform?

State conflicts in team environment. Resolution: Remote backend with locking implement kiya aur strict rule banaya — sab changes CI/CD pipeline se jayein, koi local apply nahi production mein.

> ***Interview Tip:*** *Always end with what you LEARNED and what PROCESS you IMPROVED — shows ownership and growth mindset.*

Q: How do you migrate existing infra to Terraform?

- Step 1: Existing resources inventory karo
- Step 2: HCL configuration likho existing resources match karke
- Step 3: terraform import se har resource state mein laao
- Step 4: terraform plan chalao — zero drift verify karo
- Step 5: Ab se sab changes Terraform se — koi manual change nahi

Q: How do you ensure best practices?

- Modules: har reusable pattern ke liye
- Remote backend: state locking with Azure Blob Storage
- DevSecOps: tfsec + tflint + Checkov har pipeline mein
- Code reviews: PR approval before merge
- Saved plan files: production apply with manual approval
- prevent_destroy: critical production resources pe
- Sensitive variables: secrets kabhi hardcode nahi
- Key Vault: secrets management ke liye
- Versioning + soft-delete: state storage account pe

SECTION 13 — Advanced & Tricky Concepts

Q: What is idempotency in Terraform?

Same configuration baar baar apply karo — same result milega. Agar infrastructure already desired state mein hai, kuch nahi badlega. Declarative IaC ki core property hai.

Q: Dynamic Block kya hai?

Repeated nested configuration blocks ko list ya map se dynamically generate karta hai — for example, NSG mein multiple security rules ek variable list se banana.

Q: Remote State Data Source kya hai?

Ek Terraform project doosre project ke state file se output values read kar sakta hai — VNet IDs, Key Vault URIs alag Terraform configs ke beech share karne ke liye.

> ***Interview Tip:*** *In my project, the networking module state outputs VNet ID which the VM module reads using remote state data source.*

Q: terraform plan -target kya hai?

Specific resource ke liye plan ya apply karo — sirf wo resource aur dependencies process hote hain.

Note: -target sirf debugging ke liye hai — regular workflow mein use mat karo, state inconsistencies aa sakti hain.

Q: One large state vs multiple small states?

- Multiple small states BETTER hain — har team/component apna state own kare
- Chhota blast radius — ek ki galti se doosre affect nahi hote
- Faster plan aur apply — kam resources process hote hain
- Remote state data sources se outputs share karo between states

---

SECTION ✓ Quick Reference — Last Minute Revision Card

Command / Concept	Remember This
`terraform init`	Provider download + backend setup
`terraform plan -out=tfplan`	Save plan for CI/CD approval flow
`terraform apply tfplan`	Apply reviewed plan — no surprises
`terraform state mv`	Rename resource — no destroy/recreate
`terraform state rm`	Remove from state — resource stays in Azure
`terraform import`	Bring existing resource under Terraform
`terraform force-unlock`	Stuck lock remove — carefully!
`prevent_destroy`	Critical resource ko delete se bachao
`create_before_destroy`	Zero downtime — pehle naya, phir purana delete
`ignore_changes`	Azure Policy se managed attributes ignore karo
`for_each > count`	Unique resources ke liye hamesha for_each
`sensitive = true`	Secrets ko logs mein print hone se rokta hai
`.terraform.lock.hcl`	Git mein commit karo — consistent versions
`tflint → tfsec → Checkov`	Pipeline security scan order
`Managed Identity`	Best auth — no secrets to manage or rotate

Aapka Final Mantra:

Har jawab mein real project se example do. Commands ka WHY batao, sirf WHAT nahi. Security + CI/CD mindset dikhao. 10 saal ka telecom + DevOps experience genuinely rare hai — confidently own karo!

All the Best Dilip Bhai — You've GOT This! ■